



TYCS-INS-Pract Code- Pract 1-5

Bsc. Computer Science (University of Mumbai)



Scan to open on Studocu

TYCS-SEM V

INFORMATION & NETWORK SECURITY PRACTICAL

Practical 1

Implementing Substitution and Transposition Ciphers:

Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.

Code :

(A) Ceasar Cipher

```
def encrypt(text,s):
    result = ""
    # traverse text
    for i in range(len(text)):
        char = text[i]

        # Encrypt uppercase characters
        if (char.isupper()):
            result += chr((ord(char) + s-65) % 26 + 65)
        # Encrypt lowercase characters
        else:
            result += chr((ord(char) + s - 97) % 26 + 97)
    return result

#check the above function
text=input(" Enter the text to encrypt ")
s = 3
print("Text : " + text)
str(s)
print( "Cipher: " + encrypt(text,s))
```

(B) Railfence Cipher

```
string=input("enter a string")
def RailFence(txt):
    result=""
    for i in range(len(string)):
        if(i%2==0):
            result+=string[i]
    for i in range(len(string)):
        if(i%2!=0):
            result += string[i]
    return result
print(RailFence(string))
```

Practical 2

RSA Encryption and Decryption:

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

Code:

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import binascii

# Key generation
keyPair = RSA.generate(1024)

pubKey = keyPair.publickey()
print(f'Public key: (n={hex(pubKey.n)}, e={hex(pubKey.e)})')
pubKeyPEM = pubKey.exportKey()
print(pubKeyPEM.decode('ascii'))

print(f'Private key: (n={hex(pubKey.n)}, d={hex(keyPair.d)})')
privKeyPEM = keyPair.exportKey()
print(privKeyPEM.decode('ascii'))

# Encryption
msg = 'Ismile Academy'
encryptor = PKCS1_OAEP.new(pubKey)
encrypted = encryptor.encrypt(msg.encode('utf-8'))
# Encode string to bytes
print("Encrypted:", binascii.hexlify(encrypted))
```

Practical 3

Message Authentication Codes:

Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

Code :

(A) MD 5

```
import hashlib
result = hashlib.md5(b'Ismile')
result1 = hashlib.md5(b'Esmile')
# printing the equivalent byte value.
print("The byte equivalent of hash is : ", end = "")
print(result.digest())
print("The byte equivalent of hash is : ", end = "")
print(result1.digest())
```

(B) SHA Algorithm

```
import hashlib
str = input(" Enter the value to encode ")
result = hashlib.sha1(str.encode())
print("The hexadecima equivalent if SHA1 is : ")
print(result.hexdigest())
```

Practical 4

Digital Signatures:

Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Code:

```
from Crypto.Signature import PKCS1_v1_5
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA
from Crypto import Random

# Function to generate digital signature
def generate_signature(private_key, message):
    # Load the private key
    key = RSA.importKey(private_key)
    # Generate SHA-256 hash of the message
    hashed_message = SHA256.new(message.encode('utf-8'))
    # Create a signature using the private key
    signer = PKCS1_v1_5.new(key)
    signature = signer.sign(hashed_message)
    return signature

# Function to verify the signature
def verify_signature(public_key, message, signature):
    # Load the public key
    key = RSA.importKey(public_key)
    # Generate SHA-256 hash of the message
    hashed_message = SHA256.new(message.encode('utf-8'))
    # Verify the signature using the public key
    verifier = PKCS1_v1_5.new(key)
    return verifier.verify(hashed_message, signature)

# Generate RSA key pair
random_generator = Random.new().read
key_pair = RSA.generate(2048, random_generator)
```

```
# Extract public and private keys
public_key = key_pair.publickey().export_key()
private_key = key_pair.export_key()

# Example usage
message = "Hello, World!"

# Generate a digital signature
signature = generate_signature(private_key, message)
print("Generated Signature:", signature.hex())

# Verify the digital signature
is_valid = verify_signature(public_key, message, signature)
print("Signature Verification Result:", is_valid)
```

Practical 5

Key Exchange using Diffie-Hellman:

Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

Code :

```
from random import randint

if __name__ == '__main__':
    P = 23 # A prime number
    G = 9  # A primitive root modulo P

    print('The Value of P is : %d' % P)
    print('The Value of G is : %d' % G)

    # Alice's secret number
    a = 4
    print('Secret Number for Alice is : %d' % a)

    # Alice sends this to Bob
    x = pow(G, a, P)

    # Bob's secret number
    b = 6
    print('Secret Number for Bob is : %d' % b)

    # Bob sends this to Alice
    y = pow(G, b, P)

    # Each computes the shared secret key
    ka = pow(y, a, P)
    kb = pow(x, b, P)

    print('Secret key for the Alice is : %d' % ka)
    print('Secret key for the Bob is : %d' % kb)
```